

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
FACULTAD DE CIENCIAS E INGENIERÍA

PROGRAMACIÓN 3
11ra práctica (tipo b)
(Segundo Semestre 2025)

Indicaciones Generales:

Duración: **110 minutos**.

- Se les recuerda que, de acuerdo al reglamento disciplinario de nuestra institución, constituye una falta grave copiar del trabajo realizado por otro estudiante o cometer plagio para el desarrollo de esta práctica.
- Se permite el uso de apuntes de clase, diapositivas, ejercicios prácticos y código fuente. Sin embargo, todo este material debe descargarse antes de comenzar a resolver el enunciado.
- Se permite el uso de Internet exclusivamente para consultar páginas oficiales de Microsoft y Oracle. Sin embargo, cualquier forma de comunicación con otros estudiantes o terceros está estrictamente prohibida.

Puntaje total: 20 puntos

-
- La propuesta de solución al laboratorio deberá subirse a la plataforma **Paideia** en un archivo comprimido en formato **ZIP**. No se aceptarán los trabajos compactados con otros programas como **RAR**, **WinRAR**, **7zip** o similares. Es importante asegurarse de incluir únicamente el código fuente, excluyendo el código objeto y las librerías.
 - Cada archivo deberá tener el siguiente formato de nombre “Lab11_2025_2_CO_PA_PN” donde: **CO** indica: Código del alumno, **PA** indica: Primer Apellido del alumno y **PN** indica: primer nombre. De no colocar este requerimiento se le descontará 3 puntos de la nota final. Un ejemplo de formato podría ser el siguiente: “Lab09_2025_2_19941146_Melgar_Héctor”.

Cuestionario

Pregunta 1 (5 puntos)

En este laboratorio se trabajará con una base de datos que **simula parte del registro de identificación del RENIEC**. El objetivo es desarrollar una aplicación monolítica basada en capas que permita realizar la **consulta de una persona a partir de su número de DNI**, exponiendo la funcionalidad mediante un **Servicio Web SOAP**.

Se cuenta con una base de datos que contiene la tabla **REG_PERSONAS**, la cual almacena la información básica de identificación de los ciudadanos registrados. La estructura de la tabla es la siguiente:

- **DNI**: número de documento de identidad del ciudadano. Tipo de dato **CHAR(8)**. Es la clave primaria de la tabla.
- **PATERNO**: apellido paterno del ciudadano. Tipo de dato **VARCHAR(45)**.
- **MATERNO**: apellido materno del ciudadano. Tipo de dato **VARCHAR(45)**.
- **NOMBRES**: nombres del ciudadano. Tipo de dato **VARCHAR(45)**.

Para la creación y carga inicial de datos se proporcionan los siguientes scripts:

- **script_RENIEC.sql**: crea la tabla **REG_PERSONAS** en una base de datos Microsoft SQL Server (MSSQL).

- **insert _RENIEC.sql**: inserta un conjunto de registros iniciales en la tabla **REG_PERSONAS**.

Se solicita implementar una **arquitectura monolítica basada en capas** que contenga:

- **Capa de gestión de base de datos**: manejo de la conexión y configuración del acceso a datos.
- **Capa de modelo**: definición de la clase de transferencia de datos (**PersonasDTO**) que representará los registros de la tabla.
- **Capa de persistencia y acceso a datos**: implementación del patrón *DAO*, con métodos que permitan obtener información de la base de datos.
- **Capa de reglas de negocio**: contiene la lógica que coordina el flujo de datos entre la persistencia y el servicio web.

El sistema debe permitir la **consulta de una persona a partir de su número de DNI**. Dicha consulta deberá exponerse mediante un **Servicio Web SOAP**, el cual responderá con los datos del ciudadano encontrado o con un mensaje indicando que la persona no existe en la base. El servicio debe funcionar de la siguiente manera:

- a) El cliente envía un número de DNI al servicio web.
- b) El servicio invoca la capa de negocio, la cual consulta la capa de persistencia para buscar en la base de datos.
- c) Si la persona existe, el servicio retorna un objeto con sus datos (**DNI, PATERNO, MATERNO, NOMBRES**).
- d) Si la persona no existe en la base de datos, el servicio devuelve un objeto con valores predefinidos:
 - DNI: "00000000"
 - Paterno: "NO ENCONTRADO"
 - Materno: "NO ENCONTRADO"
 - Nombres: "NO ENCONTRADO"

Este comportamiento asegura que el servicio siempre retorne una respuesta estructurada, evitando valores nulos y manteniendo la coherencia en las respuestas SOAP.

Si ejecutó correctamente lo solicitado, llame al Jefe de Práctica para que valide su avance y continúe con la siguiente pregunta. Caso contrario no podrá continuar con la pregunta 2 y esta no será evaluada.

Pregunta 2 (5 puntos)

Se solicita construir una solución en **Visual Studio Code** compuesta por **dos proyectos separados**:

- a) **Proyecto 1 – Cliente SOAP**: biblioteca que genera y expone un cliente para el servicio web desarrollado en la pregunta anterior (consulta por DNI).
- b) **Proyecto 2 – Pruebas**: módulo que *consume* el cliente del servicio desarrollado en la pregunta anterior y valida su funcionamiento. Este segundo proyecto puede implementarse como **proyecto de pruebas unitarias**.
 - La solución debe **separar responsabilidades**: el *cliente SOAP* debe encontrarse en un proyecto independiente del proyecto de *pruebas*.
 - El cliente debe consumir el **mismo WSDL** del servicio de consulta por DNI implementado anteriormente.
 - En el proyecto de pruebas se deben ejecutar, como mínimo:
 - **2 consultas** con DNIs que **existen** en la base de datos (**REG_PERSONAS**) y verificar que retorna **DNI, PATERNO, MATERNO, NOMBRES** válidos.
 - **2 consultas** con DNIs que **no existen** y verificar que retorna los **valores por omisión** definidos por la capa de negocio del servicio: **DNI = "00000000"** y **PATERNO = MATERNO = NOMBRES = "NO ENCONTRADO"**.

Si ejecutó correctamente lo solicitado, llame al Jefe de Práctica para que valide su avance y continúe con la siguiente pregunta. Caso contrario no podrá continuar con la pregunta 3 y esta no será evaluada.

Pregunta 3 (5 puntos)

En esta actividad se desarrollará un componente que simula la atención de clientes mediante una **cola de turnos**, a la cual los usuarios pueden registrarse (“pedir vez”) y avanzar conforme se atienden. La clase `ColaCliente` representa una estructura de datos en memoria que gestiona el orden de atención de los clientes. Su lógica se apoya en dos estructuras estáticas principales:

- Una lista (`ArrayList<ClienteDTO>cola`) que almacena los clientes en el orden en que fueron registrados. La clase `ClienteDTO` almacena los datos básicos del cliente registrado en la cola de atención, incluyendo el `id` o número de turno asignado, el `DNI` que identifica de forma única al cliente, así como sus apellidos (`paterno` y `materno`) y su `nombre`. Estos atributos permiten mantener la información necesaria para gestionar el orden de atención y mostrar los datos del cliente al momento de avanzar en la cola o consultar el estado de la atención.
- Un mapa (`HashMap<String, Integer>map`) que mantiene una asociación entre el **DNI del cliente** y su número de turno (`secuencia`).

La cola permite realizar dos operaciones expuestas mediante el Servicio Web **Reservas**:

- a) `pide_vez`: registra un nuevo cliente en la cola si no ha sido registrado previamente.
- b) `avanzar`: extrae (atiende) al siguiente cliente en orden de llegada.

Estructura de la clase `ColaCliente`

- `secuencia`: contador incremental que asigna un número de turno a cada nuevo cliente que pide vez.
- `cola`: lista que almacena los objetos `ClienteDTO` en el orden en que fueron añadidos.
- `map`: estructura auxiliar de tipo `Map<String, Integer>` que guarda pares clave-valor donde:
 - La **clave** es el `DNI` del cliente.
 - El **valor** es el número de turno (`secuencia`) asignado.
 - Se inicializa así `new HashMap<String, Integer>()`.

Funcionamiento del método `pide_vez`

- a) El método recibe los datos del cliente: `DNI`, `paterno`, `materno` y `nombre`.
- b) Antes de agregarlo, verifica si el `DNI` ya se encuentra en el `map` (`Boolean existe = map.containsKey(DNI)`). Esto evita que un mismo cliente solicite turno más de una vez.
- c) Si el `DNI` no existe:
 - Incrementa el contador `secuencia`.
 - Crea un nuevo objeto `ClienteDTO` con los datos del cliente y el número de turno asignado.
 - Agrega el cliente a la `cola`.
 - Registra el par (`DNI`, `secuencia`) en el `map` (`map.put(DNI, ColaCliente.secuencia)`).
- d) Finalmente, devuelve `true` si el cliente fue agregado exitosamente o `false` si ya estaba en la cola.

Funcionamiento del método `avanzar`

- a) Verifica si la `cola` está vacía. Si está vacía, retorna un objeto `ClienteDTO` con el `DNI = "00000000"` indicando que no hay clientes por atender.
- b) Si hay clientes, remueve el primer elemento de la lista (posición 0), obteniendo así el siguiente en turno.
- c) Elimina del `map` el registro asociado al `DNI` de ese cliente, ya que su turno fue atendido (`map.remove(clienteDNI)`).
- d) Retorna el objeto `ClienteDTO` correspondiente al cliente que avanzó en la cola.

Servicio Web Reservas

El servicio **Reservas** expone las operaciones de la clase **ColaCliente** a través del protocolo SOAP.

- **pide_vez**: recibe los datos del cliente como parámetros y retorna un valor booleano indicando si la solicitud fue aceptada.
- **avanzar**: retorna un objeto **ClienteDTO** correspondiente al siguiente cliente en la cola, o un objeto vacío si no hay clientes registrados.

Si ejecutó correctamente lo solicitado, llame al Jefe de Práctica para que valide su avance y continúe con la siguiente pregunta. Caso contrario no podrá continuar con la pregunta 4 y esta no será evaluada.

Pregunta 4 (5 puntos)

Se deben añadir **dos nuevos proyectos** a la solución creada en la Pregunta 2:

Proyecto A: Cliente SOAP de Cola de Clientes

- Genera el **cliente SOAP** a partir del WSDL del servicio de la **cola de clientes** (Pregunta 3).
- Expone una *fachada* para invocar al menos estas operaciones:
 - **pide_vez(DNI, paterno, materno, nombre) → Boolean**
 - **avanzar() → Cliente**
- No debe contener lógica de negocio; sólo la invocación SOAP y adaptación de tipos.

Proyecto B: Orquestador de Funcionalidades

Implementa dos funcionalidades de aplicación, **consumiendo**:

- a) el **Cliente SOAP de Cola** (Proyecto A), y
- b) el **Servicio SOAP RENIEC simulado** (Pregunta 1) para verificación de identidad.

Funcionalidad 1: Encolar clientes

- **Entrada**: DNI, paterno, materno, nombres.
- **Proceso**:
 - a) Invocar **pide_vez** del servicio de cola con los datos recibidos.
 - b) Si el servicio responde **true**, se considera que el cliente fue encolado.
 - c) Recuperar el **número de atención (secuencial)** asignado.
- **Salida**:
 - Confirmación de encolado (**true/false**).
 - Número de atención asignado.

Funcionalidad 2: Atender clientes

- **Entrada**: ninguna.
- **Proceso**:
 - a) Invocar **avanzar()** al servicio de cola para obtener el **siguiente cliente**.
 - b) Si la cola está vacía (p.ej., DNI = "00000000"), finalizar el flujo indicando *sin clientes*.
 - c) Con el DNI obtenido, consultar el **Servicio RENIEC simulado** (Pregunta 1) para recuperar **paterno, materno y nombres** oficiales.

- d) **Validar coincidencia** exacta entre los apellidos y nombres devueltos por la cola y los del servicio RENIEC.
- e) Si **coincide**: marcar como *atendido* y devolver *atención exitosa*.
- f) Si **no coincide**: *rechazar la atención* (registrar incidente) e informar la discrepancia.

■ **Salida:**

- Resultado de la atención (*atendido/rechazado/sin clientes*).
- En caso de rechazo, detalle de campos que no coinciden.

■ **Reglas:**

- La verificación con RENIEC es **obligatoria** para completar la atención.
- Si RENIEC devuelve NO ENCONTRADO o error, **no se atiende** al cliente y se informa la causa.

Si ejecutó correctamente lo solicitado, llame al Jefe de Práctica para que valide su avance. Caso contrario esta pregunta no será evaluada.

Profesores del curso: Freddy Paz Andrés Melgar
Eric Huiza

Pando, 5 de noviembre de 2025